St Francis Institute of Technology

(Autonomous Institute)

Department of Artificial Intelligence and Machine Learning

Second Year AIML Engineering (SEM-IV A.Y. 2025-26)

Web Programming Lab. Experiment Report

Experiment 10: Implement backend functionality using Express.js

1. AIM

   To create a simple Express.js server that implements a "Hello World" API, use nodemon for automatic server restarting during development, and test the API using a web browser and Postman.

2. LAB OBJECTIVE
   - 2.1. To understand the role of backend in web applications
   - 2.2. To learn the basics of Node.js
   - 2.3. To understand Express.js framework
   - 2.4. To implement routing in Express
   - 2.5. To understand middleware
   - 2.6. To use nodemon for automatic server restart
   - 2.7. To test APIs using Browser and Postman

3. LAB OUTCOME

   After completing this experiment, students will be able to:
   - 3.1. Explain the role of backend in web applications
   - 3.2. Create a basic Express server
   - 3.3. Implement simple routes
   - 3.4. Use middleware in Express
   - 3.5. Use nodemon for development efficiency
   - 3.6. Test APIs using browser/Postman

4. PREREQUISITE
   - 4.1. Basic JavaScript
   - 4.2. Command line usage
   - 4.3. HTTP methods (GET, POST)
   - 4.4. Basic understanding of client-server architecture

5. THEORY

   5.1. Role of Backend in Web Applications In a web application:

   - 5.1.1. Frontend → User Interface (HTML, CSS, React, etc.)
   - 5.1.2. Backend → Handles:
     - Business logic
     - Database operations
     - Authentication
     - API responses
   - 5.1.3. Backend responsibilities:
     - Receive client requests
     - Process data
     - Send responses
     - Interact with databases

5.2. Overview of Node.js

5.2.1. Node.js is:
A JavaScript runtime environment
Built on Chrome's V8 engine
Used for building server-side applications

5.2.2. Advantages:
Non-blocking I/O
Event-driven architecture
Fast and scalable

5.3. What is Express.js?

5.3.1. Express.js is:
A minimal and flexible Node.js web framework · Used for building APIs and web servers

5.3.2. Features:
Routing
Middleware support
HTTP utility methods
Simplified server creation

5.4. Express Routing

Routing refers to:
Defining how the server responds to client requests.

Example:
GET / → Returns homepage
GET /api → Returns API response

5.5. Middleware in Express

Middleware functions:
Execute during request-response cycle
Can modify request/response
Can log, authenticate, validate, etc.

Examples:
Logging middleware
JSON parser middleware

5.6. nodemon

nodemon is:
A development tool
Automatically restarts server when files change

Benefits:
No need to manually restart server
Improves productivity

6. PROCEDURE

6.1. Install Node.js

6.1.1. Download and install Node.js from official website.

6.1.2. Verify installation using:

- node -v
- npm -v

```
C:\Users\Angel>node -v
v24.13.1

C:\Users\Angel>npm -v
11.8.0
```

6.2. Create Project Folder

6.2.1. Create a new folder for backend project.

6.2.2. Open the folder in terminal.

6.2.3. Initialize project using npm init.

6.2.4. Generate package.json file.

```
C:\Users\Angel>cd exp10

C:\Users\Angel\exp10>npm init -y
Wrote to C:\Users\Angel\exp10\package.json:

{
  "name": "exp10",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

6.3. Install Express

6.3.1. Install Express using npm.

6.3.2. Confirm it is added in dependencies.

```
C:\Users\Angel\exp10>npm install express

added 65 packages, and audited 66 packages in 5s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
  "dependencies": {
    "express": "^5.2.1"
  }
```

6.4. Create Basic Server

6.4.1. Create a file named server.js.
6.4.2. Import Express.
6.4.3. Create an Express application.
6.4.4. Define a port number.
6.4.5. Start the server using listen method.
6.4.6. Verify server is running in terminal.

```
C:\Users\Angel\exp10>node server.js
Server running at http://localhost:3000
```

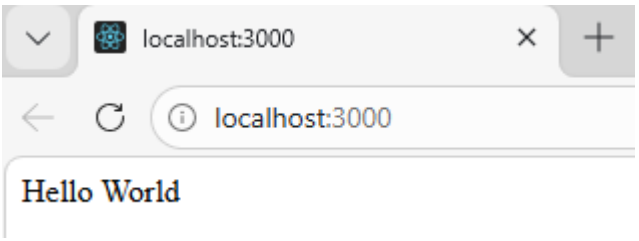
6.5. Create "Hello World" Route

6.5.1. Define a GET route for root path (/).
6.5.2. Send a response "Hello World".
6.5.3. Save file and restart server.
6.5.4. Open browser and visit:
- http://localhost:PORT

6.5.5. Verify response appears in browser.

server.js

```
const express = require("express");    // Import Express
const app = express();                 // Create Express
application
const PORT = 3000;                     // Define port number
// Hello World Route
app.get("/", (req, res) => {
    res.send("Hello World");
});
app.listen(PORT, () => {               // Start server
    console.log(`Server running at http://localhost:${PORT}`);
});
```



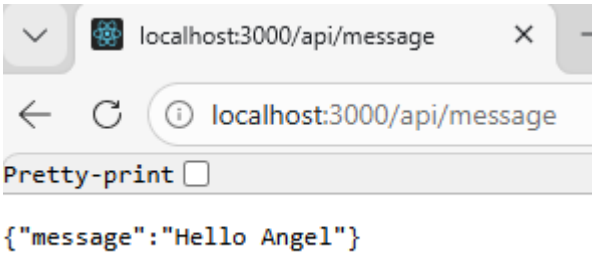
6.6. Create Simple API Route

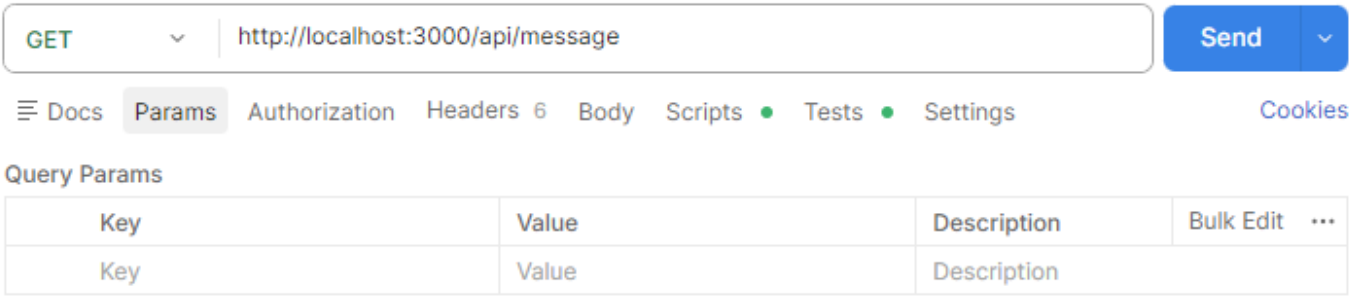
6.6.1. Create another route:
- Example: /api/message

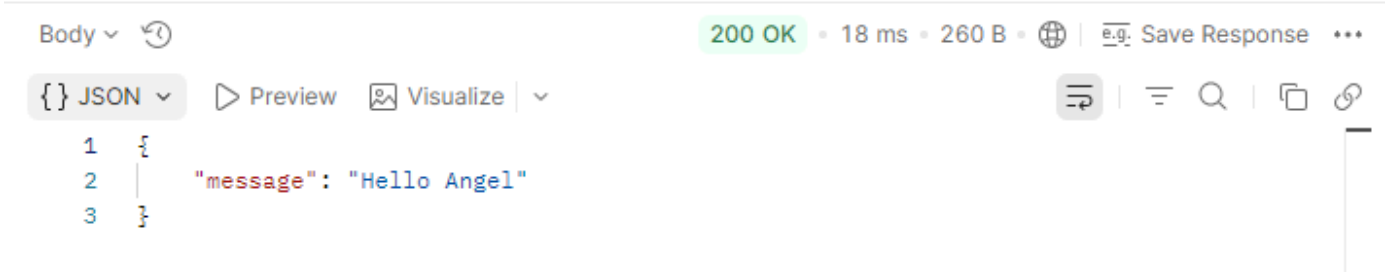
6.6.2. Send JSON response.
6.6.3. Test using:
- Browser
- Postman

server.js

```
// API Route
app.get("/api/message", (req, res) => {
    res.json({
        message: "Hello Angel"
    });
});
```







6.7. Implement Middleware

6.7.1. Add middleware function.

6.7.2. Log request method and URL.

6.7.3. Observe middleware execution in terminal.

server.js

```
// Middleware
app.use((req, res, next) => {
    console.log("Request Method:", req.method);
    console.log("Request URL:", req.url);
    next();
});
```

```
C:\Users\Angel\exp10>node server.js
Server running at http://localhost:3000
Request Method: GET
Request URL: /
Request Method: GET
Request URL: /api/message
```

6.8. Install and Use nodemon

6.8.1. Install nodemon globally or as dev dependency.

6.8.2. Run server using nodemon instead of node.

6.8.3. Modify server file.

6.8.4. Observe automatic restart without manual intervention.

```
C:\Users\Angel\exp10>nodemon server.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Server running at http://localhost:3000
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
Server running at http://localhost:3000
Request Method: GET
Request URL: /
Request Method: GET
Request URL: /api/message
```

6.9. Test Using Postman

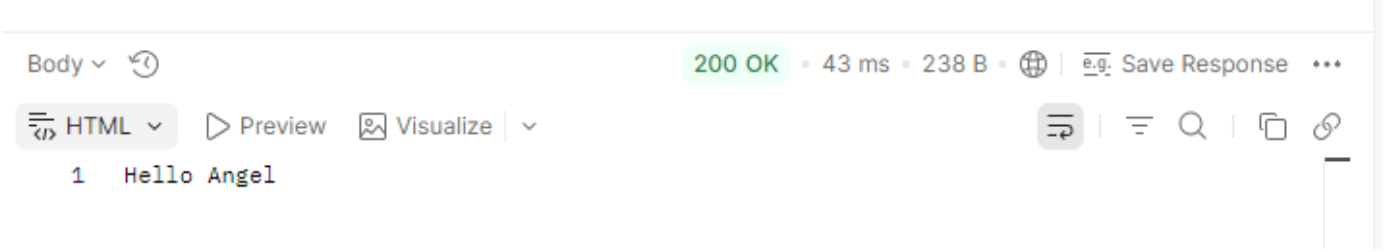
6.9.1. Open Postman.

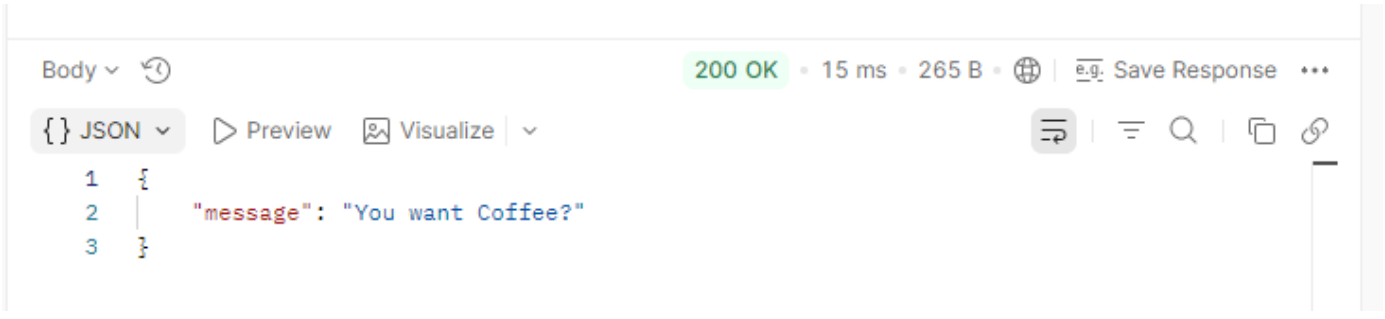
6.9.2. Send GET request to:

- http://localhost:PORT/
- http://localhost:PORT/api/message

6.9.3. Verify response status and body.

- verify:
  Server starts successfully
  Routes respond correctly
  JSON response is displayed
  Middleware logs appear
  nodemon auto-restarts server





7. RESULTS / OUTCOME EXPECTED

7.1. Does the Express server start successfully and listen on the specified port without errors?
7.2. When accessing the root route ("/"), does the “Hello World” API respond correctly in the browser?
7.3. Do the defined API routes return proper JSON responses when accessed?
7.4. Is the middleware function executed during each client request (e.g., logging request details in the terminal)?
7.5. Does nodemon automatically restart the server whenever changes are made to the source file?
7.6. Are the APIs tested successfully using a browser or Postman with correct status codes and responses?

8. CONCLUSION

8.1. What is the role of backend in a web application?
8.2. Why is Express used with Node.js?
8.3. What is routing in Express?
8.4. What is middleware?
8.5. Why is nodemon used during development?

9. POST-EXPERIMENT QUESTIONS

9.1. What is the difference between frontend and backend?
9.2. What happens if port is already in use?
9.3. What is the difference between GET and POST?
9.4. Can multiple middleware functions be used?
9.5. How does Express handle requests and responses?

10. REFERENCES

10.1. Node.js Official Documentation – https://nodejs.org
10.2. Express.js Documentation – https://expressjs.com
10.3. Postman Documentation
10.4. MDN Web Docs – HTTP Methods